

CS3520 LIMO Group Project: Milestone 1 (LIMO Instruction Set Architecture)

National University of Lesotho · CS3520 Computer Organisation and Architecture · AY 2026/2027, Semester A

Team: _____ · Repository: limo-_____ · Release tag: m1 · Document version 1.0

AI assistance used in drafting this document is declared in AI_USE.md.

A1 Style and format

LIMO is a 32-bit load-store machine derived from RV32I. Only bala (load) and beha (store) touch memory; every other instruction works on registers. Memory is byte-addressed, little-endian and word-aligned (word addresses are multiples of 4). Every instruction is exactly 32 bits, so the PC advances by 4. Instruction memory and data memory are separate.

LIMO keeps the RISC-V style by using the same field positions, opcode and funct values, and immediate layouts for the I and S formats. The main difference is which instruction formats are included.

Format	Used by	Included	Rationale
R	kopanya, tlosa, kapa, le	Yes	Register-register arithmetic and logic.
I	eketse, suthisa, bala	Yes	Immediate arithmetic, shift by constant, load.
S	beha	Yes	Store: two registers and an offset.
B	none	No (team decision)	Replaced by the S layout, see below.
J	none	No (team decision)	Optional in the handout; no jump instruction is needed for the 10 chosen instructions.
U	none	No	Optional in the handout; constants are built with eketse and suthisa.

Departure 1: branches use the S layout. lekanya and phapanyo have two source registers and one offset, which is exactly what the S layout carries. They are distinguished from beha by their own opcode

(1100011). The offset is a 12-bit signed byte offset (a multiple of 4), relative to the address of the branch itself.

- Why: the RV32I B-type scrambles the offset bits ($\text{imm}[12|10:5]$ and $\text{imm}[4:1|11]$) so that its offset also encodes bit 0 and bit 12. LIMO instructions are always 4-byte aligned, so those bits are wasted. With the S layout there is a single, unscrambled immediate generator shared by I, S and branch instructions, which is easier to build, draw and explain.
- Cost: branch reach is ± 2 KiB (512 instructions) instead of ± 4 KiB. The Ripes cross-check is still done at assembly level (same label, same behaviour).

Departure 2: no J format, no jal or jalr. An unconditional jump can be written as a pseudo-instruction, `lekanya noto, noto, label` (`r0` always equals `r0`, so the branch is always taken). Pseudo-instructions are expanded by the assembler and are not counted among the 10 instructions.

Why keep RV32I field positions at all? Every LIMO instruction is then a valid RV32I instruction restricted to registers `x0` to `x15`, so each program can be translated to RV32I and checked in Ripes, and the decoder is the standard textbook decoder.

A2 Registers

Register file size: 16 registers. Register numbers need 4 bits instead of the 5 bits of RV32I. To keep every field in its RV32I position, the top bit of each register field is reserved and must be 0.

Quantity	RV32I (32 registers)	LIMO (16 registers)
Bits per register number	5	4
Unused (reserved) bits per R-type instruction	0	3 (bits 11, 19, 24)
Register file storage	$32 \times 32 = 1024$ bits	$16 \times 32 = 512$ bits
Read-port multiplexer	32:1	16:1
Forwarding comparators (<code>rs1</code> , <code>rs2</code> against EX/MEM.rd and MEM/WB.rd)	5-bit	4-bit
Registers available for data (after <code>r0</code> , <code>r1</code> , <code>r2</code>)	29	13

Rationale: a smaller register file means narrower comparators and multiplexers in the hazard unit, which makes the datapath easier to draw and the forwarding logic easier to explain. The price is register pressure and three unused encoding bits (see A6a).

General-purpose and special-purpose registers

Number	Sesotho name	Short names	Role	Rationale for the name
r0	noto	noto	Hard-wired zero (writes ignored)	Team choice.
r1	lehae	lehae	Return address	Lehae means home: where a call returns to.
r2	lesupa	lesupa	Stack pointer	Lesupa: the one that points.
r3 to r8	nakoana	n1 to n6	Temporaries (not preserved)	Nakoana: short-lived, temporary.
r9 to r12	kenyo	k1 to k4	Arguments and results	Kenyo: what is put in.
r13 to r15	poloko	p1 to p3	Saved registers (preserved)	Poloko: keeping, safekeeping.
PC	sesupo	sesupo	Program counter. Not in the register file and not numbered; only branches change it.	Programmer-visible special register.

The assembler accepts either the number (r3) or any Sesotho name (n1). Note that r1 (return address) and r2 (stack pointer) are defined by convention; LIMO v1.0 has no instruction that writes r1 automatically because it has no jump-and-link.

A3 Instruction set

LIMO uses 10 instructions.

Mnemonic	Class	Format	Operation	Rationale
kopanya	Arithmetic	R	$rd = rs1 + rs2$	Basic addition.
tlosa	Arithmetic	R	$rd = rs1 - rs2$	Subtraction; also used to compare.
eketse	Arithmetic (immediate)	I	$rd = rs1 + imm$	Constants, counters, pointers, register copy.
kapa	Logic	R	$rd = rs1 \text{ OR } rs2$	Bitwise OR.

Mnemonic	Class	Format	Operation	Rationale
le	Logic	R	$rd = rs1 \text{ AND } rs2$	Bitwise AND, masking.
suthisa	Shift	I	$rd = rs1$ shifted left by shamt	Multiply by powers of 2; builds large constants.
bala	Load	I	$rd = \text{Mem}[rs1 + \text{imm}]$ (word)	The only way to read memory.
beha	Store	S	$\text{Mem}[rs1 + \text{imm}] = rs2$ (word)	The only way to write memory.
lekanya	Branch	Branch (S layout)	if $rs1 = rs2$ then $PC = PC + \text{offset}$	Equality test and loops.
phapanyo	Branch	Branch (S layout)	if $rs1 \neq rs2$ then $PC = PC + \text{offset}$	Inequality test and loops.

Coverage of the handout minimum: 3 arithmetic with one immediate (kopanya, tlosa, eketse); 3 logic with one shift (kapa, le, suthisa); a word load (bala) and word store (beha); 2 conditional branches (lekanya, phapanyo); no multiply, divide, floating point, CSRs or exceptions. Total 10, within the 10 to 20 allowed. suthisa shifts left logically by a constant of 0 to 31.

A4 Sesotho assembly

Glossary

Mnemonic	Sesotho meaning	English	Syntax	Operation	RV32I equivalent
kopanya	to join together	add	kopanya rd, rs1, rs2	$rd = rs1 + rs2$	add
tlosa	to remove	subtract	tlosa rd, rs1, rs2	$rd = rs1 - rs2$	sub
eketse	to add more	add immediate	eketse rd, rs1, imm	$rd = rs1 + \text{imm}$	addi
bala	to read	load word	bala rd, imm(rs1)	$rd = \text{Mem}[rs1 + \text{imm}]$	lw
beha	to put, to	store word	beha rs2,	$\text{Mem}[rs1 +$	sw

Mnemonic	Sesotho meaning	English	Syntax	Operation	RV32I equivalent
	place		imm(rs1)	imm] = rs2	
lekanya	to measure, to compare	branch if equal	lekanya rs1, rs2, label	if rs1 = rs2 branch	beq
phapanyo	difference	branch if not equal	phapanyo rs1, rs2, label	if rs1 != rs2 branch	bne
kapa	or	bitwise OR	kapa rd, rs1, rs2	rd = rs1 OR rs2	or
le	and	bitwise AND	le rd, rs1, rs2	rd = rs1 AND rs2	and
suthisa	to cause to move	shift left	suthisa rd, rs1, shamt	rd = rs1 << shamt	slli

Register mapping to RV32I: rN corresponds to xN (only x0 to x15 are used).

Assembler rules. Mnemonics and register names are case-insensitive and use only the letters a to z (no diacritics), so they can be typed on a standard keyboard. Comments start with #. Labels are written name: at the start of a line. immediates are decimal or hexadecimal (0x...). Memory operands are written offset(base). Branch targets are labels; the assembler computes the byte offset and reports an error if it is not a multiple of 4 or is outside -2048 to +2044.

Apostrophe policy. In Lesotho orthography the apostrophe is part of a letter (ts', ch', and similar), so it changes the word.

1. No LIMO mnemonic or register name contains an apostrophe, so all instructions are plain ASCII.
2. The assembler treats the straight apostrophe ' (U+0027) and the typographic apostrophes (U+2018 and U+2019) as the same character and normalises them to U+0027. This makes text pasted from word processors or phones work.
3. An apostrophe is allowed inside labels and is significant: a label spelt with ts' is a different label from the same word spelt with ts. This respects Sesotho spelling instead of silently deleting a letter.
4. The assembler has no character literals, so the apostrophe never clashes with other syntax.

A5 Encoding and specification

Format diagrams (bit positions; 0 marks a reserved bit that must be 0)

```

bit  31..25  24 23..20 19 18..15 14..12 11 10..7 6..0
R   funct7  0 rs2  0 rs1  funct3  0 rd   opcode
I   imm[11:0] (bits 31..20) 0 rs1  funct3  0 rd   opcode
I-shift funct7  0 sham(4:0 in bits 24..20) 0 rs1 funct3 0 rd opcode
S/Branch imm[11:5] 0 rs2  0 rs1  funct3 imm[4:0] (bits 11..7) opcode

```

In the S and branch layout, bits 11..7 hold imm[4:0], so bit 11 is not reserved there. The immediate generator has only two cases: I-type imm = sign-extend(inst[31:20]); S and branch imm = sign-extend(inst[31:25], inst[11:7]).

Opcode and funct assignments (identical to RV32I)

Instruction	Format	opcode	funct3	funct7
kopanya	R	0110011	000	0000000
tlosa	R	0110011	000	0100000
kapa	R	0110011	110	0000000
le	R	0110011	111	0000000
eketse	I	0010011	000	none
suthisa	I (shift)	0010011	001	0000000
bala	I	0000011	010	none
beha	S	0100011	010	none
lekanya	Branch (S layout)	1100011	000	none
phapanyo	Branch (S layout)	1100011	001	none

Immediate ranges

Field	Width	Range	Notes
I and S immediate (eketse, bala, beha)	12 bits, signed	-2048 to 2047	Sign-extended to 32 bits.
Shift amount (suthisa)	5 bits	0 to 31	
Branch offset	12 bits, signed, multiple of 4	-2048 to +2044	Relative to the branch's own address: target = address of branch + offset. Reach is 512 instructions backward or forward.

Load and store addresses must be multiples of 4. Behaviour for misaligned addresses or non-zero reserved bits is undefined (LIMO has no exceptions).

Hand-encoded instructions

Instruction	Binary (by field)	Hex
kopanya r3, r1, r2 (R)	0000000 00010 00001 000	0x002081B3

Instruction	Binary (by field)	Hex
	00011 0110011	
eketse r3, r1, 5 (l)	00000000101 00001 000 00011 0010011	0x00508193
suthisa r3, r1, 2 (l-shift)	0000000 00010 00001 001 00011 0010011	0x00209193
bala r3, 4(r4) (l load)	00000000100 00100 010 00011 0000011	0x00422183
beha r3, 8(r4) (S)	0000000 00011 00100 010 01000 0100011	0x00322423
lekanya r3, r4, +8 (Branch)	0000000 00100 00011 000 01000 1100011	0x00418463

Each binary string is written as in the diagram, with the reserved top bit of each register field included in its 5-bit group. For lekanya, rs1 = r3, rs2 = r4 and the offset +8 is split as imm[11:5] = 0000000 and imm[4:0] = 01000. For forward branches below 2048 bytes this value happens to match the RV32I beq encoding; for backward branches the two encodings differ, because LIMO does not scramble the offset.

A6 Design decisions

(a) What does the register-file size cost and buy?

Sixteen registers need 4-bit register numbers, so each of rd, rs1 and rs2 has one reserved bit (three per R-type instruction) while the field positions stay identical to RV32I. It buys a 512-bit register file instead of 1024, 16:1 read multiplexers, and forwarding comparators 4 bits wide instead of 5. It costs register pressure: with r0, r1 and r2 reserved, 13 registers remain for data. Evidence: our three sample programs use at most seven registers (r3 to r9), so pressure is not yet visible, but nested loops with saved values would reach the limit.

(b) Which registers are architectural, which microarchitectural?

Architectural (visible to programs): the PC, r0 to r15, and instruction and data memory.

Microarchitectural (hidden): the IR, the pipeline registers IF/ID, ID/EX, EX/MEM and MEM/WB, the control signals, and the hazard and forwarding units. The ISA must not specify them because it is a contract between software and every possible implementation. Evidence: Program 1 ends with r3 = 15 and Mem[0] = 15 on a single-cycle machine (18 cycles) or on our pipeline (30 cycles); only timing differs. If the ISA fixed the pipeline registers, no implementation could change its number of stages without breaking existing programs.

(c) Where are branches resolved, and what does it cost?

LIMO resolves branches in EX, where the ALU compares rs1 and rs2 and the target is computed. A taken branch flushes the two younger instructions in IF and ID: Program 1 loses $4 \times 2 = 8$ of its 30 cycles,

Program 2 loses 4 of its 33. Resolving in ID would flush only one instruction, but needs a dedicated comparator, extra forwarding paths into ID, and a one-cycle stall when a branch directly follows the instruction producing its operand (eketse then phapanyo in Program 1), which cuts the gain. We chose EX: no extra hardware, and the same ALU path as arithmetic.

(d) Why does a load followed by a dependent instruction still stall under full forwarding?

The loaded data exists only at the end of MEM, but a dependent instruction needs it at the start of its EX. Without a stall, kopanya would be in EX during cycle 4, while bala is still reading memory, so no forwarding wire can supply the value. One bubble is needed; kopanya then receives the value from MEM/WB in cycle 5.

Cycle	1	2	3	4	5	6	7
bala n5, 0(n2)	IF	ID	EX	MEM	WB		
kopanya n4, n4, n5		IF	ID	ID (stall)	EX	MEM	WB
bubble				EX	MEM	WB	

Evidence: Program 2 has one such stall per iteration (3 in total). Moving eketse n2, n2, 4 between the two instructions would remove them.

(e) Would LIMO benefit from a flags register?

RISC-V branches compare two registers themselves, so no hidden state links an arithmetic instruction to a later branch. A flags register would let one tlosa feed several branches and would make less-than tests cheap, but every arithmetic instruction would write it, creating a new RAW dependency through the flags. The flags would need their own forwarding and become extra state to save. LIMO has only lekanya and phapanyo, which compare in one instruction (Program 1's loop test needs no setup), so flags would add hazards for no saving. A signed less-than branch would give magnitude tests more cheaply.

(f) As programs grow, what breaks first?

Our estimate, to be confirmed by measurement: the immediate range breaks first, then register count, then branch reach. Without a U-type instruction, eketse reaches only -2048 to 2047, so larger constants and addresses need a sequence of eketse and suthisa instructions and an extra register. Next, 13 usable registers limit nested loops. Branch reach of +/-2 KiB (512 instructions) breaks last, since our longest loop body is four instructions. Evidence: our programs only use constants and addresses from 0 to 30 and need none of these limits yet.

Sample LIMO programs

Assumptions for the expected cycle counts: five stages, full forwarding, one stall for a load followed by a dependent instruction, branches resolved in EX with predict-not-taken (a taken branch flushes 2 instructions), 4 cycles to fill the pipeline. Register numbers: n1 to n6 = r3 to r8, k1 = r9. All results are hand-derived and must still be cross-checked in Ripes after translating to RV32I.

Program 1: sum of 1 to 5

```
eketse n1, noto, 0    # n1 = sum = 0
eketse n2, noto, 5    # n2 = counter = 5
loop: kopanya n1, n1, n2    # sum = sum + counter
      etetse n2, n2, -1    # counter = counter - 1
      phapanyo n2, noto, loop # repeat while counter != 0
      beha n1, 0(noto)    # Mem[0] = sum
```

Expected: r3 = 15, r4 = 0, Mem[0] = 15. Instructions executed: 18. Branch taken 4 times (8 flushed).
Cycles: 18 + 4 + 8 = 30.

Program 2: sum of a three-element array

```
eketse n1, noto, 10
beha n1, 0(noto)    # Mem[0] = 10
eketse n1, noto, 20
beha n1, 4(noto)    # Mem[4] = 20
eketse n1, noto, 30
beha n1, 8(noto)    # Mem[8] = 30
eketse n2, noto, 0    # n2 = pointer
eketse n3, noto, 12    # n3 = end address
eketse n4, noto, 0    # n4 = sum
loop: bala n5, 0(n2)    # n5 = Mem[pointer]
      kopanya n4, n4, n5    # sum = sum + n5 (load-use stall)
      etetse n2, n2, 4    # pointer = pointer + 4
      phapanyo n2, n3, loop # repeat until pointer = end
      beha n4, 12(noto)    # Mem[12] = sum
```

Expected: r3 = 30, r4 = 12, r5 = 12, r6 = 60, r7 = 30; Mem[0] = 10, Mem[4] = 20, Mem[8] = 30, Mem[12] = 60. Instructions executed: 22. Branch taken 2 times (4 flushed), 3 load-use stalls. Cycles: 22 + 4 + 3 + 4 = 33.

Program 3: logic, shift, memory and a taken branch

```
eketse n1, noto, 12    # n1 = 1100b = 12
eketse n2, noto, 10    # n2 = 1010b = 10
le n3, n1, n2    # n3 = 1000b = 8
kapa n4, n1, n2    # n4 = 1110b = 14
tlosa n5, n4, n3    # n5 = 14 - 8 = 6
suthisa n6, n5, 2    # n6 = 6 << 2 = 24
beha n6, 0(noto)    # Mem[0] = 24
bala k1, 0(noto)    # k1 = 24
lekanya k1, n6, ok    # equal, so branch is taken (load-use stall)
eketse n6, noto, 99    # skipped
ok: beha k1, 4(noto)    # Mem[4] = 24
```

Expected: r3 = 12, r4 = 10, r5 = 8, r6 = 14, r7 = 6, r8 = 24, r9 = 24; Mem[0] = 24, Mem[4] = 24 (the instruction etetse n6, noto, 99 never executes). Instructions executed: 10. One load-use stall, one taken branch (2 flushed). Cycles: 10 + 4 + 1 + 2 = 17.